
1. Iteration Method (Fixed-Point Iteration)

The **Iteration Method**, also known as Fixed-Point Iteration, is a technique for finding the roots of an equation. A root is a value x that makes an equation $f(x)=0$ true.

Explanation

The core idea is to first rearrange the equation $f(x)=0$ into the form $x=g(x)$. A solution to this new equation is called a "fixed point" because it's a point that the function $g(x)$ doesn't change. This fixed point is also the root of the original equation.

The process is as follows:

1. Start with an **initial guess**, x_0 .
2. Generate the next guess by plugging the current guess into the function $g(x)$. The formula is: $x_{n+1}=g(x_n)$.
3. Repeat this process. If the method works (converges), the sequence of values $(x_1, x_2, x_3, \dots)$ will get closer and closer to the actual root.

Analogy: Imagine you're looking for a specific spot on a map. You have a rule that says, "From your current position, move to this new calculated position." You start somewhere random (initial guess), apply the rule to find a new spot, apply the rule again from there, and so on. If the rule is good, you'll spiral or zigzag right onto the target spot.

Example

Find a root of the equation $f(x)=x^2-x-2=0$.

1. **Rearrange** to the form $x=g(x)$. One way is $x^2=x+2$, which gives $x=x+2$. So, our iteration function is $g(x)=x+2$.
2. Make an **initial guess**. Let's start with $x_0=1$.
3. **Iterate:**
 - $x_1=g(x_0)=1+2=3 \approx 1.732$
 - $x_2=g(x_1)=1.732+2=3.732 \approx 1.932$

- $x_3 = g(x_2) = 1.932 + 2 = 3.932 \approx 1.983$
- $x_4 = g(x_3) = 1.983 + 2 = 3.983 \approx 1.996$

The values are getting closer and closer to **2**, which is the actual root of the equation ($22 - 2 - 2 = 0$).

Key Words (Bengali Translation)

- **Iteration:** পুনরাবৃত্তি (Punrabritti)
 - **Root:** মূল (Mul)
 - **Equation:** সমীকরণ (Shomikoron)
 - **Initial Guess:** প্রাথমিক অনুমান (Prathomik Onuman)
 - **Convergence:** অভিসৃতি (Obhisriti)
 - **Fixed-Point:** স্থির বিন্দু (Sthir Bindu)
-

2. Bisection Method

The **Bisection Method** is a reliable root-finding algorithm that is guaranteed to work for any continuous function.

Explanation

This method is based on the Intermediate Value Theorem, which states that if a continuous function has values of opposite signs at the two ends of an interval, it must cross the x-axis (i.e., have a root) somewhere within that interval.

The process is as follows:

1. Choose an **interval** $[a, b]$ such that $f(a)$ and $f(b)$ have **opposite signs**.
2. Calculate the **midpoint**, $c = \frac{a+b}{2}$.
3. Evaluate $f(c)$.
4. If $f(c)$ has the same sign as $f(a)$, the new interval becomes $[c, b]$.
5. If $f(c)$ has the same sign as $f(b)$, the new interval becomes $[a, c]$.
6. Repeat this process, halving the interval each time, until the interval is small enough for

the desired accuracy.

Analogy: It's like finding a specific word in a dictionary. You open to the middle page. If your word comes after the words on that page, you know it's in the second half of the book, so you discard the first half. You then take the remaining half and open it to its middle page. You repeat this, always cutting your search area in half until you find the word.

Example

Find the root of $f(x)=x^3-4x-9=0$.

1. **Choose an interval.** Let's try $a=2$ and $b=3$.
 - $f(2)=2^3-4(2)-9=8-8-9=-9$ (Negative)
 - $f(3)=3^3-4(3)-9=27-12-9=6$ (Positive)The signs are opposite, so a root exists between 2 and 3.
2. **Iteration 1:**
 - Midpoint: $c=(2+3)/2=2.5$.
 - $f(2.5)=(2.5)^3-4(2.5)-9=15.625-10-9=-3.375$ (Negative).
 - Since $f(2.5)$ is negative, it replaces $f(2)$. The new interval is **[2.5,3]**.
3. **Iteration 2:**
 - Midpoint: $c=(2.5+3)/2=2.75$.
 - $f(2.75)=(2.75)^3-4(2.75)-9\approx 20.797-11-9=0.797$ (Positive).
 - Since $f(2.75)$ is positive, it replaces $f(3)$. The new interval is **[2.5,2.75]**.

This process continues until the root is found to the desired precision (the actual root is approx. 2.7065).

Key Words (Bengali Translation)

- **Bisection:** দ্বিখণ্ডন (Dwikhondon)
 - **Interval:** ব্যবধান (Byabodhan)
 - **Midpoint:** মধ্যবিন্দু (Moddhobindu)
 - **Continuous Function:** অবিচ্ছিন্ন ফাংশন (Obichhinno Function)
 - **Opposite Signs:** বিপরীত চিহ্ন (Biporit Chinho)
-

3. False Position Method (Regula Falsi)

The **False Position Method** is a root-finding algorithm that is often faster than the Bisection Method.

Explanation

Like the Bisection Method, it starts with an interval $[a,b]$ where $f(a)$ and $f(b)$ have opposite signs. However, instead of just taking the midpoint, it takes a "smarter" guess. It draws a straight line (a **chord**) between the points $(a,f(a))$ and $(b,f(b))$ on the curve. The new guess for the root, c , is the point where this line crosses the x -axis. This usually gets closer to the real root faster than the midpoint does.

The formula for finding c is:

$$c = \frac{f(b) - f(a)}{f(b) - f(a)} \cdot b - \frac{f(b)}{f(b) - f(a)} \cdot a$$

The interval is then updated in the same way as the Bisection Method.

Analogy: If the Bisection Method is like walking halfway across a valley to find the lowest point, the False Position Method is like standing on one hill, looking at the other, and estimating where the lowest point *should* be based on the slopes. It's an educated guess.

Example

Let's use the same equation: $f(x) = x^3 - 4x - 9 = 0$ with the interval $[2,3]$.

We know $a=2, b=3, f(a)=-9, f(b)=6$.

1. Iteration 1:

- Calculate c :
 $c = \frac{6 - (-9)}{6 - (-9)} \cdot 3 - \frac{6}{6 - (-9)} \cdot 2 = \frac{15}{15} \cdot 3 - \frac{6}{15} \cdot 2 = 3 - 0.8 = 2.2$
- $f(2.2) = (2.2)^3 - 4(2.2) - 9 = 10.648 - 8.8 - 9 = -7.152$ (Negative).
- The new interval is **[2.2, 3]**.

2. Iteration 2:

- Now $a=2.2, b=3, f(a)=-7.152, f(b)=6$.
- Calculate new c :
 $c = \frac{6 - (-7.152)}{6 - (-7.152)} \cdot 3 - \frac{6}{6 - (-7.152)} \cdot 2.2 = \frac{13.152}{13.152} \cdot 3 - \frac{6}{13.152} \cdot 2.2 = 3 - 1.064 \approx 1.936$

Notice that after just two iterations, we are already much closer to the root (2.7065) than with

the Bisection Method.

Key Words (Bengali Translation)

- **False Position:** মিথ্যা অবস্থান (Mithya Obosthan)
 - **Chord:** জ্যা (Jya)
 - **X-intercept:** এক্স-অক্ষ ছেদক (X-okkho Chhedok)
-

5. Gauss Elimination Method

Gauss Elimination is a standard method for solving a **system of linear equations**.

Explanation

The method works by transforming the system of equations into a simpler form that is easy to solve.

The process has two main parts:

1. **Forward Elimination:** The system of equations is first written as an **augmented matrix**. Then, using elementary **row operations** (swapping rows, multiplying a row by a constant, adding a multiple of one row to another), the matrix is converted into an **upper triangular form**. This means all the elements below the main diagonal are zero. This effectively eliminates one variable at a time from the equations.
2. **Back Substitution:** Once the matrix is in upper triangular form, the last equation has only one variable and can be solved directly. This solution is then substituted back into the second-to-last equation to find the next variable, and so on, working your way back up to the first equation.

Analogy: It's like solving a complex puzzle by simplifying it one step at a time. You first use the rules to solve for one piece, then use that piece's position to figure out the next one, and continue until the whole puzzle is solved.

Example

Solve the system:

$$x+y+z=6$$

$$x-y+z=2$$

$$2x+y-z=1$$

1. Augmented Matrix:

$$1121-1111-1621$$

2. **Forward Elimination:**

○ $R_2 \rightarrow R_2 - R_1$:

$$1021-2110-16-41$$

○ $R_3 \rightarrow R_3 - 2R_1$:

$$1001-2-110-36-4-11$$

○ $R_3 \rightarrow R_3 - 0.5R_2$:

$$1001-2010-36-4-9 \text{ (Upper Triangular Form)}$$

3. **Back Substitution:**

○ From Row 3: $-3z = -9 \Rightarrow z = 3$.

○ From Row 2: $-2y + 0z = -4 \Rightarrow -2y = -4 \Rightarrow y = 2$.

○ From Row 1: $x + y + z = 6 \Rightarrow x + 2 + 3 = 6 \Rightarrow x = 1$.

The solution is $(x, y, z) = (1, 2, 3)$.

Key Words (Bengali Translation)

- **Gauss Elimination:** গাউসীয় অপসারণ (Gausiyo Oposharon)
- **System of Linear Equations:** রৈখিক সমীকরণ জোট (Roikhik Shomikoron Jot)
- **Augmented Matrix:** বিবর্ধিত ম্যাট্রিক্স (Bibordhito Matrix)
- **Upper Triangular Form:** উর্ধ্ব ত্রিভুজাকার রূপ (Urdho Tribhujakar Rup)
- **Back Substitution:** পশ্চাৎ প্রতিস্থাপন (Poshchat Protisthapon)
- **Row Operations:** সারি অপারেশন (Shari Operation)

6. Gauss-Jordan Elimination

Gauss-Jordan Elimination is an extension of Gauss Elimination for solving systems of linear equations.

Explanation

This method takes the forward elimination process one step further. Instead of stopping at an upper triangular matrix, it continues applying row operations until the matrix is in **reduced row echelon form**. This form has two properties:

1. It is in upper triangular form.
2. The leading non-zero entry in each row is a 1, and it's the only non-zero entry in its column.

The result is that the coefficient part of the augmented matrix becomes the **identity matrix** (1s on the diagonal, 0s everywhere else). The major advantage is that this **eliminates the need for back substitution**. The solution to the system can be read directly from the last column of the final matrix.

Analogy: If Gauss Elimination is like solving a puzzle and leaving the last few logical steps for you to do in your head, Gauss-Jordan is a method that continues the systematic process until the puzzle is completely and explicitly solved for you.

Example

Using the same upper triangular matrix from the Gauss Elimination example:

1001-2010-36-4-9

1. **Make leading entries 1:**

- $R_2 \rightarrow R_2 / (-2)$
- $R_3 \rightarrow R_3 / (-3)$

100110101623

2. **Make entries above leading 1s zero:**

- $R_1 \rightarrow R_1 - R_3$:

100110001323

- $R1 \rightarrow R1 - R2$:

100010001123(Reduced Row Echelon Form)

3. Read the Solution:

The final matrix corresponds to the equations:

$$1x + 0y + 0z = 1 \Rightarrow x = 1$$

$$0x + 1y + 0z = 2 \Rightarrow y = 2$$

$$0x + 0y + 1z = 3 \Rightarrow z = 3$$

Key Words (Bengali Translation)

- **Gauss-Jordan Elimination:** গাউস-জর্ডান অপসারণ (Gaus-Jordan Oposharon)
 - **Reduced Row Echelon Form:** লঘুকৃত সারি ইশেলন রূপ (Loghukrito Shari Echelon Rup)
 - **Identity Matrix:** অভেদ ম্যাট্রিক্স (Obhed Matrix)
-

8. Least Squares Regression

Least Squares Regression is a statistical method used to find the **best-fit line** that describes the relationship between a dependent variable (y) and an independent variable (x).

Explanation

When you have a set of data points on a scatter plot, they rarely form a perfect straight line. The goal of linear regression is to find the one line that comes closest to all the points simultaneously.

The "least squares" method does this by minimizing the **sum of the squared residuals**. A residual is the vertical distance between an actual data point (y) and the point predicted by the regression line ($\hat{y}$). We square these distances to ensure they are positive and to give more weight to points that are far from the line. The method uses calculus to find the slope (m) and intercept (c) of the line $\hat{y} = mx + c$ that make this sum of squared errors as small as possible.

Analogy: Imagine your data points are a collection of stars in the sky. Least squares

regression is like using a laser pointer (the line) and trying to aim it so that the average distance from the laser beam to all the stars is minimized.

Example

Find the best-fit line for the data points (x,y): (1,3),(2,5),(3,4).

1. **Calculate necessary sums:**

- $n=3$ (number of points)
- $\Sigma x=1+2+3=6$
- $\Sigma y=3+5+4=12$
- $\Sigma x^2=1^2+2^2+3^2=1+4+9=14$
- $\Sigma xy=(1)(3)+(2)(5)+(3)(4)=3+10+12=25$

2. Calculate slope (m) and intercept (c) using the formulas:

$$m = \frac{n(\Sigma xy) - (\Sigma x)(\Sigma y)}{n(\Sigma x^2) - (\Sigma x)^2}$$

$$c = \bar{y} - m\bar{x} \text{ (where } \bar{x} = \frac{\Sigma x}{n} \text{ and } \bar{y} = \frac{\Sigma y}{n} \text{)}$$

- $m = \frac{3(25) - (6)(12)}{3(14) - (6)^2} = \frac{42 - 72}{42 - 36} = \frac{-30}{6} = -5$
- $\bar{x} = 6/3 = 2$ and $\bar{y} = 12/3 = 4$
- $c = 4 - (-5)(2) = 4 + 10 = 14$

The best-fit line is $\hat{y} = -5x + 14$.

Key Words (Bengali Translation)

- **Least Squares:** লঘিষ্ঠ বর্গ (Loghishtho Borgo)
- **Regression:** নির্ভরণ (Nirbhoron)
- **Best-fit line:** সর্বোত্তম মানানসই রেখা (Sorbottam Manansoi Rekha)
- **Residual / Error:** অবশেষ / ত্রুটি (Obshesh / Truti)
- **Dependent Variable:** অধীন চলক (Odhin Cholok)
- **Independent Variable:** স্বাধীন চলক (Shadhin Cholok)

9. Logistic Regression

Logistic Regression is a statistical method used for **classification** tasks. Unlike linear

regression which predicts a continuous number, logistic regression predicts the **probability** of a categorical outcome (e.g., Yes/No, Spam/Not Spam).

Explanation

The core of logistic regression is the **sigmoid (or logistic) function**, which has an "S" shape. This function takes any real-valued number as input and squashes it into a value between 0 and 1, which can be interpreted as a probability.

The process is:

1. A linear equation is created from the input features (e.g., $z = b_0 + b_1x_1 + b_2x_2$).
2. The result, z , is passed through the sigmoid function: $P = 1 + e^{-z}$.
3. The output, P , is the predicted probability. For example, if we are predicting whether an email is spam, a P of 0.92 would mean there is a 92% probability that the email is spam.
4. A threshold (usually 0.5) is used to make a final classification. If $P > 0.5$, we classify it as "Yes" (or 1); otherwise, "No" (or 0).

Analogy: Imagine a doctor deciding if a patient has a certain disease based on test results. Linear regression might give a "sickness score" from -100 to +100, which is hard to interpret. Logistic regression takes the same test results and gives a clear probability, like "85% chance of having the disease," which is much more useful for making a diagnosis (a classification).

Example

Predict if a student will pass (1) or fail (0) based on hours studied (x).

Let's say a trained logistic regression model found the coefficients to be $b_0 = -4$ and $b_1 = 1$. The model is:

$$P(\text{Pass}) = 1 + e^{-(-4 + 1x)}$$

Now, let's predict the outcome for a student who studied for **3 hours**.

1. Calculate z : $z = -4 + 1(3) = -1$.
2. Apply the sigmoid function:
 $P(\text{Pass}) = 1 + e^{-(-1)} = 1 + e^1 \approx 1 + 2.7181 \approx 0.269$.

The model predicts a ~27% probability of passing. Since this is less than our 50% threshold, we would classify the student as likely to **fail**.

Key Words (Bengali Translation)

- **Logistic Regression:** লজিস্টিক নির্ভরণ (Logistic Nirbhoron)
 - **Classification:** শ্রেণীবিন্যাস (Srenibinnas)
 - **Probability:** সম্ভাবনা (Shombhabona)
 - **Sigmoid Function:** সিগময়েড ফাংশন (Sigmoid Function)
 - **Categorical Outcome:** বর্গীয় ফলাফল (Borgiyo Folafol)
-

11. Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a powerful **optimization algorithm** used to train machine learning models, including both linear and logistic regression. Its goal is to find the model parameters (like slope and intercept) that minimize the model's error, which is measured by a **cost function**.

Explanation

Imagine a graph where the "low points" or "valleys" represent small errors and the "high points" or "hills" represent large errors. Gradient Descent tries to find the lowest point in this valley.

- **Gradient:** The gradient is a vector that points in the direction of the steepest ascent (uphill).
- **Descent:** To minimize the error, we take steps in the direction **opposite** to the gradient (downhill). The size of each step is determined by a **learning rate**.

The key difference between types of gradient descent is how much data is used to calculate the gradient for each step:

- **Batch Gradient Descent:** Uses the *entire dataset* to calculate the gradient. It's accurate but very slow on large datasets.
- **Stochastic Gradient Descent (SGD):** Uses just **one randomly selected data point** to calculate the gradient and take a step. It is much faster but the path it takes is noisy and erratic.
- **Mini-Batch Gradient Descent:** A compromise that uses a small, random batch of data

points. This is the most common approach.

Analogy: You are on a foggy mountain and want to get to the lowest point.

- **Batch GD:** You stop, check the slope in every direction around you, decide on the best way down, and take one careful step.
- **SGD:** You only check the slope right under your foot and immediately take a quick step in that downhill direction. You'll stumble around a lot, but you'll get down the mountain much faster.

Example

Let's train a model $y=mx$ to fit the point $(x,y)=(4,10)$, starting with a guess of $m=1$.

- **Cost Function (Error):** $J=(y_{\text{actual}}-y_{\text{predicted}})^2=(10-m \cdot 4)^2$.
- **Gradient** (derivative of J with respect to m): $\frac{dJ}{dm}=2(10-4m)(-4)=-8(10-4m)$.
- **Learning Rate (α):** Let's use 0.01.

Step 1:

- Current $m=1$.
- Calculate gradient at $m=1$: $\frac{dJ}{dm}=-8(10-4(1))=-8(6)=-48$.
- Update m : $m_{\text{new}}=m_{\text{old}}-\alpha \cdot \text{gradient}=1-(0.01)(-48)=1+0.48=1.48$.

Step 2:

- Current $m=1.48$.
- Calculate gradient at $m=1.48$: $\frac{dJ}{dm}=-8(10-4(1.48))=-8(10-5.92)=-8(4.08)=-32.64$.
- Update m : $m_{\text{new}}=1.48-(0.01)(-32.64)=1.48+0.3264=1.8064$.

The process repeats. With each step, the value of m gets closer to the optimal value of **2.5** (since $10=2.5 \times 4$).

Key Words (Bengali Translation)

- **Stochastic:** দৈব (Doibo)
- **Gradient:** নতি (Noti)
- **Descent:** অবতরণ (Obotoron)
- **Optimization:** সর্বোত্তমকরণ (Sorbottamkoron)
- **Cost Function:** ব্যয় ফাংশন (Byay Function)

- **Learning Rate:** শেখার হার (Shekhar Haar)
- **Parameters:** পরামিতি (Poramiti)